Visualizing Data with Gnuplot

Roychowdhury & Banerjee

1 Data Files and Curve Fitting

In *gnuplot* Functions and Data are two complementary types of objects; while data are available either from experiments or program runs; fitted curves, represented by functions, describe the trends implicit in the data. The fitting process in *gnuplot* is very simple; you need to have a file (the 'datafile') from which the data comes and a function, defined in a separate file, also called the 'fitting script'. The data used by *gnuplot* does not come from a spreadsheet but usually from text files where the data is written in columns. The function is declared with initially undetermined parameters like $a, b, c \ldots$ (the 'fitting parameters') as in `f(x)=a+sin(b*x+c)`. If you try to plot the function right now then it fails, simply because *gnuplot* do not know about the *numerical* values of the fitting parameters at this stage. Then a fitting command, also present in the fitting script is run and may typically look like:

```
fit f(x) 'datafile_dat.txt' via a,b,c
```

Where `datafile_dat.txt` represents the name of the datafile (with extension `txt`). The command is equivalent to the literal phrase: *Fit the function $f(x)$ to the datafile by finding numerical values of a, b and c.* After this process is over and if the fitting process is successful all the fitting parameters get numerical values. You can print their values out or plot the function together with the data for a better understanding of the dataset. The *art* of fitting however, consists of supplying carefully constrained *initial* values to some of the parameters and / or picking up sections of the data (say, the linear regime) appropriate for fitting. It can be aquired and refined only through interest and iteration. Note that *gnuplot* uses a very powerful nonlinear least squares algorithm for fitting data and its results can be reliably quoted for scientific communication. In the following, we supply the necessary details for understanding the format of data files and the curve fitting process.

1.1 Facts

1. *gnuplot* reads data from text files. The data is expected to be numerical and to be stored in the file in whitespace-separated columns. Lines beginning with a hashmark (#) are considered to be comment lines and are ignored. A typical datafile looks like

```
#x   y
1.2  2.3
2.4  4.5
3.2  5.6
4.5  7.6
```

   The canonical way to think about this is that the x-value is in column 1 and the y-value is in column 2. If there are additional y-values corresponding to each x-value, they are listed in subsequent columns. However, if there is only one column in the datafile, it is taken to be a y column. In that case, the x-values are taken to belong to an index set consisting of integers, starting from 0.

©Suparna Roychowdhury and Shibaji Banerjee: All rights for circulation, modification, utilization and (re)distribution of this document or parts thereof lies solely with the copyright holders and authors of this document.

2. To plot a datafile, use its name

```
gnuplot>plot [0:4] [0:8] 'simple1_dat.txt'
```

The method used in the previous factsheet on plotting 2-d functions, using block variables is not suitable for lots of data, specially when they can change! Reserve their use for marking salient points.

3. Since data files typically contain many different data sets, we'll usually want to select the columns to be used as x and y-values. This is done through the using directive to the `plot` command:

```
gnuplot>plot 'simple1_dat.txt' using 1:2
```

4. If there are multiple data sets in one datafile one appended to one another, *gnuplot* can handle such data sets by plotting them as different graphs. For *gnuplot* , blank lines in a data file are significant. A set of double blank lines is used to distinguish data sets in a file. Each set can be addressed in the `plot` command as if it were in a separate file using the `index` directive to plot (*index* starts from 0).

```
gnuplot>plot 'simple_dat.txt' index 0 using 1:2 w linespoints, \
>'simple_dat.txt' index 1 using 1:2 w linespoints
```

5. Override default plotting style for data sets and adding title

```
gnuplot>plot [0:4] [0:8] 'simple1_dat.txt' title 'some data' with lines
```

6. One way of joining the datapoints by a smooth curve is to use the **smooth** option. The smooth directive takes an additional parameter, which must be one of the following: `unique, frequency, bezier, sbezier, csplines,acsplines`, each one employing a different smoothing algorithm (you can find out more about these smoothing options from any book on numerical techniques).

```
gnuplot>plot [0:4] [0:8] 'simple1_dat.txt' smooth csplines
```

7. *gnuplot* includes a facility to perform nonlinear least-square fits to data. The basic workflow for fitting data with a function consists of three steps: define a function, then use it as argument to fit, and finally plot the function together with the data set using plot. In the following we provide an example for an exponential fit. Note that initial values have been supplied to the fitting parameters for better control over the fitting process.

```
gnuplot>f(x) = a*exp(-b*x)
gnuplot>a=200
gnuplot>b=1
```

©Shibaji Banerjee and Suparna Roychowdhury, See Page 1 for full text of copyright notice

```
gnuplot>fit f(x) 'sample_dat.txt' u 1:2 via a,b
gnuplot>plot 'sample.txt',f(x)
gnuplot>print a,b
```

The marker `u` in the line with the `fit` command stands for the option `using`. `using` provides the option of fitting a data-set with errorbars ie. `u 1:2:3` would mean that the 3rd column of the dataset contains y-errors. The y-errors can represent experimental uncertainties or error bounds on individual values of y. The code for fitting a data-set with y-errorbars would be as follows:

```
gnuplot>f(x) = a*exp(-b*x)
gnuplot>a=200
gnuplot>b=1
gnuplot>fit f(x) 'sample.txt' u 1:2:3 via a,b
gnuplot>plot 'sample_dat.txt',f(x)
gnuplot>print a,b
```

8. Styling: To plot exptl. data with error bars, one uses the command 'yerr' or 'xerr' or 'xyerr' depending on the data having only errors in y or errors in x or errors in both. The datafile has to be in the format of *column-1: x-data, column 2: y-data, column 3: xlow-data, column 4: xhigh-data, column 5: ylow-data and column-6: yhigh-data* or *column-1: x-data, column 2: y-data, column 3: dy-data.*

```
gnuplot>plot 'datafile_dat.txt' using 1:2:3 w yerrorbars        # ( x, y, dy )
gnuplot>plot 'datafile_dat.txt' using 1:2:3:4:5:6 w xyerrorbars # ( x, y, xlow,
                                                                # xhigh, ylow, yhigh )
```

9. Two files are typically used in the fitting runs: `filename_dat.txt` (data file), `filename_plt.txt` (plotting script). It is important to keep in mind the separate identities of these files.

Exercise 1

(a) Create a datafile `simple1_dat.txt` containing the following data

```
2.3
1.2
3.5
6.9
```

Plot the data. Note that the data are plotted against their index numbers in the file. Replot the data set so that one can see data point markers joined by segmented lines.

©Shibaji Banerjee and Suparna Roychowdhury, See Page 1 for full text of copyright notice

(b) Replot the same dataset. Use segmented lines in one plot to join the data-points in one trace. In another trace, join the datapoints by a smooth curve. Explore the other smoothing options (like `frequency` or `bezier`) too.

(c) Plot the following dataset (name of file: `sample_dat.txt`), choosing appropriate bounds.

```
1.0 200.6
2.0 100.3
3.0 43.2
4.0 25.6
5.0 8.9
```

Fit the above dataset, using a suitable function. Plot the dataset along with the function and examine visually, the nature of the fit. Note that the history of the fit process is kept in a `.log` file. Find out more about the fit process by reading the associated help and jot down the facts that you find useful. Compare the representation of the dataset using point markers, segmented-lines, smooth-curve, and fitted-curve all in a single plot.

(d) You have been supplied with a data-file called `cavendish_dat.txt`. Note that there are two data sets in the data file written sequentially since these two data-sets were taken by two different people on two different days for two different time intervals. Each data block is separated by 2 blank lines. The experimental data points are stored in three columns - x, y and dy. The uncertainties are given as absolute errors in y-values. Fit the above data-set with a sinusoidal curve with exponential decay, $\theta(t) = \theta_0 + a\exp(-t/\tau)\sin(2\pi t/T + \phi)$. The various fitting parameters are a, the amplitude of oscillations τ, the period of oscillations ϕ, the initial phase T, the exponential decay time and θ_0 being the shift from the equilibrium position.

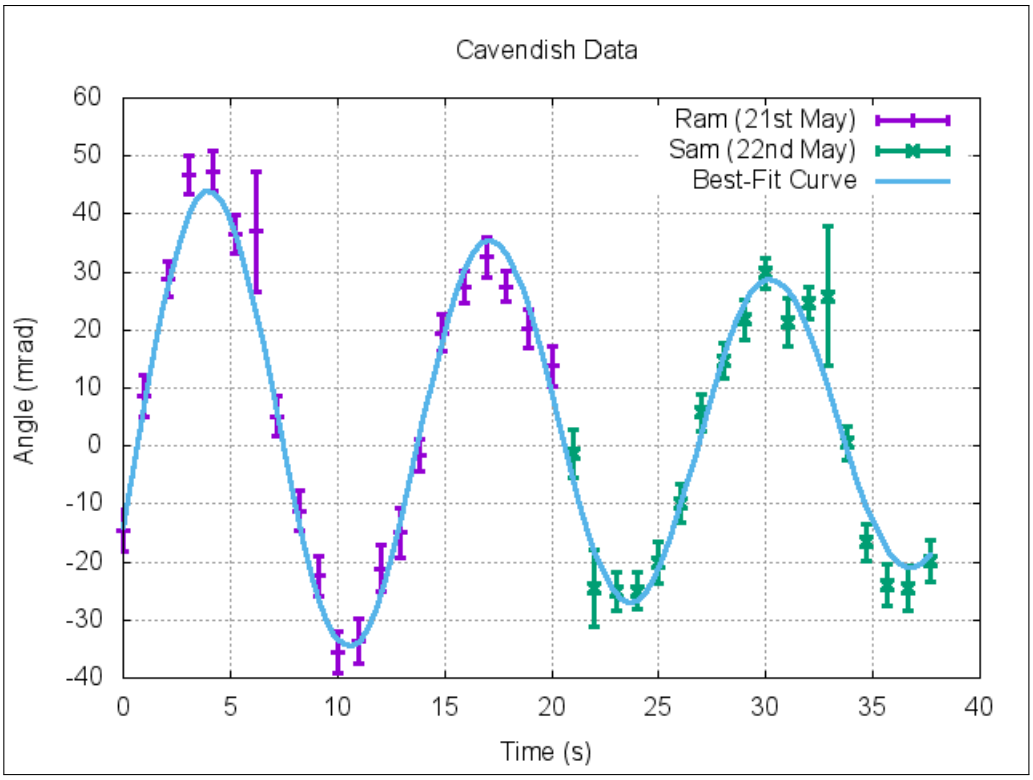
(a) Plot the two data-sets with the '*index*' syntax in a single graph along with the errorbars.

(b) Fit the whole data-set with y-errorbars using the syntax given above in the facts.

(c) Plot the data-set with the fit overlayed on it.
NOTE: An excerpt of the essential lines of the code for fitting and plotting the data-set is given below. The figure given below is an indicative output plot. Try and match your output plot with this figure.

```
theta(x)=theta0 + a * exp(-x / tau) * sin(2 * pi * x/ T + phi)
fit theta(x) "cavendish_dat.txt" using 1:2:3 via theta0, a, tau, T, phi
plot "cavendish_dat.txt" ind 0 u 1:2:3 title "Ram (21st May)" with yerrorbars lw 3,\
"cavendish_dat.txt" ind 1 u 1:2:3 title "Sam (22nd May)" with yerrorbars lw 3,\
theta(x) title "Best-Fit Curve" lw 3
```

©Shibaji Banerjee and Suparna Roychowdhury, See Page 1 for full text of copyright notice



©Shibaji Banerjee and Suparna Roychowdhury, See Page 1 for full text of copyright notice